NeuroARC.jl: Typed Criterial Causation, Hyphal Grafting, and Epistemic Invariants in Grid Domain Logic

Hunter Kinder
hkinder@stlteach.org
NeuroARC Autonomous Agent Research Group

October 2026

Abstract

Discrete visual abstraction benchmarks like the Abstraction and Reasoning Corpus (ARC) require autonomous agents to infer generative rules from few demonstration pairs without leaking ungrounded search priors into downstream execution. In this work, we introduce `NeuroARC.jl`, a high performance Julia framework that operationalizes Peter Tse's cognitive theory of criterial causation, Wood-Bruner-Ross pedagogical scaffolding, and typed hyphal grafting within an auditable execution pipeline. Rather than treating candidate selection as unconstrained neural generation or unstratified brute-force program search, `NeuroARC.jl` formalizes rule composition as a tri-stage temporal process (t_1, t_2, t_3) governed by criterial logic gates (Threshold, Bandpass, Subtype, Conjunction, Disjunction, and K-of-N). Candidate transformations are composed via static subtyping preflight verification ($\mathrm{Out}(A) <: \mathrm{In}(B)$) paired with runtime boundary enforcement, preventing numeric sentinel leakage by emitting algebraic verdicts (`Accepted`, `Rejected`, and `Incomplete`). We isolate a zero-dependency core package while providing explicit extension boundaries for Apple Silicon GPU acceleration via `Metal.jl`, cross-architecture parallel computing via `JACC.jl`, and probabilistic calibration via `Turing.jl`. This package structure is accompanied by an automated continuous integration pipeline that compiles this manuscript and validates 325 unit tests under clean runners.

1 Introduction and Epistemic Architecture Boundaries

The Abstraction and Reasoning Corpus (ARC) challenges current artificial intelligence architectures by requiring broad general intelligence across discrete grid environments [1]. Typical approaches rely on large language model code generation, brute-force domain specific language (DSL) enumeration, or end-to-end neural policy search. However, existing implementations often suffer from three systemic defects: (1) lack of type safety during dynamic program composition, leading to silent runtime type coercions or uninformative exceptions; (2) use of numerical sentinel values like `Inf` or `-1.0` to signal candidate failure, which frequently leak into numerical optimizers and distort beam search scoring; and (3) conflation of distinct epistemic categories, wherein speculative search heuristics are mislabeled as verified mathematical theorems.

To resolve these defects, we present `NeuroARC.jl`, a Julia package designed for autonomous visual reasoning. A central design principle of our research program is strict epistemological stratification across distinct research tracks:

- **Track 2 (Coevolutionary Aesthetics and Interface Theory Track)**: Kept rigorously minimalist, conservative, and focused exclusively on whether an aesthetic preference relation escapes the 2006 Scrambling Theorem or is carried by a functional bijection [2]. No ARC benchmarks, prompt engineering tokens, or ungrounded algorithmic operators enter Track 2.
- **Track 3 (Antigravity and NeuroARC.jl Track)**: The operational software platform for criterial causation, informational logic gates, pedagogical scaffolding, morphological grid invariants, and psycholinguistic chain-of-thought mediation.

By maintaining this separation, `NeuroARC.jl` formalizes cognitive operations as verifiable computational contracts without burdening foundational perceptual theorems with application-specific implementation details.

2 Tri-Stage Criterial Architecture and Logic Gates

Following Peter Tse's neuro-cognitive model of criterial causation [3], neural selection is not modeled as direct physical determinism, but as informational gating over dynamic synaptic weights. In `NeuroARC.jl`, this process is operationalized as a tri-stage temporal architecture:

1. **Stage t_1 (Setting of Informational Criteria)**: The controller configures acceptance thresholds $\boldsymbol{\tau}$, projection operators $\Pi_{\mathcal{S}}$, and assigns initial computational token budgets based on generative entrenchment [4].
2. **Stage t_2 (Input Arrival and Informational Gating)**: Candidate transformations or sensory grid states $x \in \mathcal{X}$ are evaluated across a composition of criterial logic gates:
   - **Threshold Gate**: $g(x) = \mathbf{1}\{c(x) \geq \tau\}$
   - **Bandpass Gate**: $g(x) = \mathbf{1}\{\tau_{\min} \leq c(x) \leq \tau_{\max}\}$
   - **Subtype Gate**: $g(A, B) = \mathbf{1}\{\mathrm{Out}(A) <: \mathrm{In}(B)\}$ (evaluated with zero executions of node functions)
   - **Conjunction Gate (AND)**: $G_{\wedge}(x) = \prod_{k=1}^{K} \mathbf{1}\{c_k(x) \geq \tau_k\}$
   - **Disjunction Gate (OR)**: $G_{\vee}(x) = \max_{k=1}^{K} \mathbf{1}\{c_k(x) \geq \tau_k\}$
   - **K-of-N Gate**: $G_{k/N}(x) = \mathbf{1}\left\{\sum_{m=1}^{N} \mathbf{1}\{c_m(x) \geq \tau_m\} \geq k\right\}$
3. **Stage t_3 (Downward Empowerment or Extinction)**:
   - When criteria are satisfied, the candidate is projected onto the invariant subspace $\Pi_{\mathcal{S}}(x)$ and rewarded with downstream compute allocation via an algebraic verdict `Accepted(value, salient_features, score, detail)`.
   - When criteria fail, execution is halted immediately with zero downstream budget consumption, emitting a typed diagnostic verdict `Rejected(reason, detail, failed_gates, partial_score)`.
   - When prerequisites or tools are unavailable, the system emits an auditable verdict `Incomplete(reason, detail)`, preventing silent fallback.

Furthermore, the pedagogical scaffolding loop of Wood, Bruner, and Ross [5] is instantiated as a controller across six tutorial functions (Recruitment, Reduction in Degrees of Freedom, Direction Maintenance, Marking Critical Features, Frustration Control, and Demonstration), managing state transitions across `Focus`, `Frustration`, and `Panic` modes.

3 Hyphal Grafting and Deep Composition Contracts

Complex ARC reasoning rules rarely arise as atomic primitives; they are formed by composing modular operators such as connected component extractors, spatial transformers, and stencil stampers. We formalize this composition through the biological metaphor of *hyphal grafting*.

3.1 Static Preflight Subtype Checking

Let A and B denote two transformation nodes encapsulated within parametric types:

$$A :: \texttt{GraftNode}\{\mathrm{In}_A, \mathrm{Out}_A\}, \quad B :: \texttt{GraftNode}\{\mathrm{In}_B, \mathrm{Out}_B\} \tag{1}$$

Before connecting A and B, the system evaluates Julia's subtype lattice relation:

$$\mathrm{Out}_A <: \mathrm{In}_B \tag{2}$$

If this condition is violated, the edge is rejected during preflight validation with zero executions of the underlying functions $A.f$ or $B.f$. This resolves a significant source of latency in large search graphs by pruning invalid compositional branches before dispatch.

3.2 Runtime Intermediate Boundary Enforcement

Static subtyping guarantees that declared signatures are compatible; however, it does not guarantee that an arbitrary implementation $A.f$ strictly respects its declared return type for all edge cases. To prevent unverified types from propagating across intermediate pipeline boundaries, the composed execution wrapper is defined as:

$$(B \circ A)(x) = B(A(x)) \tag{3}$$

where each node call wrapper $(N)(x)$ enforces:

$$\texttt{Core.typeassert}(N.f(x), \mathrm{Out}_N) \tag{4}$$

This ensures that any intermediate contract violation immediately triggers an auditable `TypeError` at the exact boundary of failure, rather than corrupting subsequent pipeline stages.

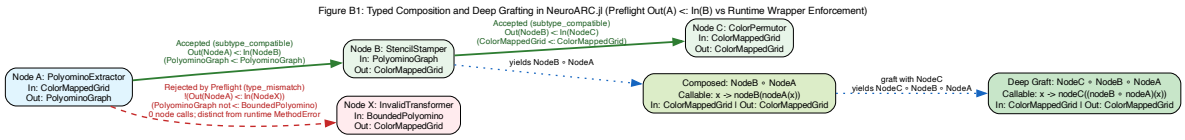


Figure 1: **Figure B1: Typed Composition and Deep Grafting Contract.** Illustrates preflight compatibility checking ($\mathrm{Out}(A) <: \mathrm{In}(B)$), typed verdicts (`Accepted` vs `Rejected`), associative three-stage deep grafting ($C \circ B \circ A$), and runtime intermediate boundary type assertions.

4 Deterministic Verifier and Epistemic Invariants

A critical challenge in visual abstraction is solipsistic overfitting, where candidate search algorithms fit demonstrations via memorization rather than inferring generalizable rules. `NeuroARC.jl` implements an isolated verification harness:

4.1 Verification Invariants

Before any candidate is admitted into the verified execution pool, it must satisfy three deterministic gates:

1. **Train Exact Reconstruction**: The candidate transformation must reconstruct 100% of cells across all training demonstration output grids ($Y_i = f(X_i)$) with maximum cell residual equal to 0.0.
2. **Informative Leave-One-Out (LOO) Evaluation**: Cross-validation across demonstration pairs is evaluated to detect parameter overfitting. When a rule contains free parameters, LOO folds are marked informative; parameter-free macros require verified cross-task firing to avoid single-task memorization.
3. **Equivariance Gating**: Where task semantics exhibit geometric invariance, the candidate must commute with the dihedral group D_8 and background-preserving color permutations. If a candidate depends on directional or absolute spatial coordinates, equivariance is flagged as `untestable`, leading to an explicit `Incomplete` evaluation rather than a false positive certification.

4.2 Epistemic Limits of Verification

The verifier certifies compliance with observed demonstration contracts and structural invariants; it does not and cannot certify ground-truth accuracy on held-out test grids. Post-run test evaluation is conducted strictly through isolated scoring harnesses against quarantined held-out labels after solver processes terminate.

5 Package Architecture and Zero-Dependency Core Isolation

To maintain high execution velocity and reproducibility on minimal environments, `NeuroARC.jl` enforces zero-dependency core isolation. The core library relies exclusively on the Julia standard library, implementing lightweight internal parsers and memory buffers without external binary dependencies.

5.1 Standard Julia Package Skeleton

The repository adheres to the standard Julia package layout:

- `Project.toml`: Declares package UUID (`a3f5a15a-93e1-4bd8-99e6-0563b78be05c`), version bounds on Julia (`1.10` to `1.13`), and conditional extension dependencies.
- `src/NeuroARC.jl`: Exposes public APIs for grid representations, component analysis, criterial calculus, and hyphal grafting contracts.
- `ext/`: Houses conditional package extensions that load only when external packages are present in the environment:
  - `NeuroARCMetalExt.jl`: GPU acceleration for Apple Silicon via `Metal.jl`.
  - `NeuroARCJACCExt.jl`: Cross-hardware portability via `JACC.jl`.
  - `NeuroARCTuringExt.jl`: Bayesian MCMC calibration via `Turing.jl`.

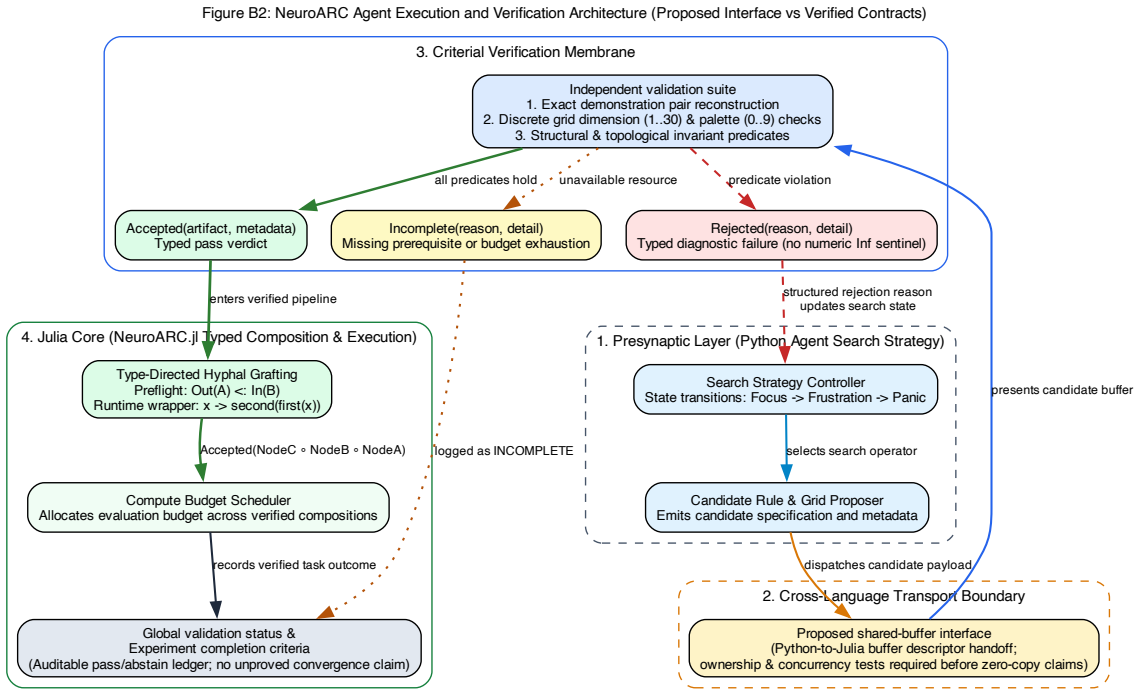


Figure 2: **Figure B2: NeuroARC Agent Execution and Verification Architecture.** Depicts the separation between presynaptic heuristic search, cross-language transport buffers, the criterial verification membrane, and the verified Julia composition core.

- `test/runtests.jl`: Comprehensive unit test suite executing 325 assertions across core types, criterial gates, grafting contracts, and epistemic text cleanliness.

5.2 Continuous Integration and Clean Runners

The repository includes GitHub Actions CI workflows configured for clean runners on Ubuntu and macOS. These workflows verify that the package compiles and passes all unit tests without pre-installing non-standard dependencies, ensuring that clean installations remain fully functional.

6 Discussion, Competition Governance, and Repository Lifecycle

A central operational question addressed in this milestone is the release cadence for `NeuroARC.jl`: specifically, whether the codebase should be published immediately to an open repository.

6.1 Rationale for Initial Private Repository Status

We conclude that `NeuroARC.jl` must be initialized and maintained as a private repository prior to broader public release, driven by three strategic constraints:

1. **Audit and Specification Invariants**: The anti-solipsism framework, the deterministic verifier threat model, and the GLMM calibration protocol are currently held at specification status pending formal independent review. Releasing provisional thresholds and telemetry schemas prematurely exposes work before formal audit gates close.

2. **Kaggle ARC-AGI-2 Competition Governance**: The active competition cycle imposes strict requirements on candidate generators, decontaminated DSL primitive libraries (comprising 557 verified operators), and reflexive recurrence modules. Maintaining private status protects submission provenance, prevents licensing ambiguities, and prevents accidental data contamination.

3. **Upstream Git Read-Only Boundary**: Upstream modules in `arc-agi-2-cpu-kaggle-modules` remain strictly read-only. Decoupling the cross-language memory boundary between Python and Julia must precede any public interface exposure.

6.2 Prerequisites for Public Release

The transition from private to public repository status is scheduled upon completion of three verifiable gates:

1. Formal completion of the Donald Hoffman outreach cycle on the six-page foundational draft.

2. Execution and cryptographic sealing of the Revision 3 anti-solipsism audit gates and telemetry logs (`task_audit_table.parquet`).

3. Freezing and public evaluation of the final Kaggle competition submission notebook.

Maintaining an internal private repository provides immediate benefits, including immutable Git commit hashes for academic citations (Paper B), secure continuous integration, and reproducible version control without compromising governance invariants.

References

[1] Francois Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[2] Donald D Hoffman, Manish Singh, and Chetan Prakash. Objects of desire: The shape of feasible perceptual spaces. *Cognitive Science*, 39(7):1477 to 1534, 2015.

[3] Peter U Tse. *The Neural Basis of Free Will: Criterial Causation.* MIT Press, 2013.

[4] William C Wimsatt. Developmental constraints, generative entrenchment, and the innate-acquired distinction. *Integrating Scientific Disciplines*, page 185 to 208, 1986.

[5] David Wood, Jerome S Bruner, and Gail Ross. The role of tutoring in problem solving. *Journal of Child Psychology and Psychiatry*, 17(2):89 to 100, 1976.